💡 Proposed Solution Template

FlightFinder - Comprehensive Solution Design

1. Solution Overview

FlightFinder: Intelligent Flight Booking Platform

Vision: To create the most user-friendly, transparent, and secure flight booking platform that simplifies travel planning for customers while empowering operators with efficient management tools.

Mission: Eliminate booking complexity and provide a seamless experience through innovative technology, transparent pricing, and role-based functionality.

Core Value Proposition: Book flights in 3 simple steps with complete price transparency, secure transactions, and effortless management - all powered by modern MERN stack technology.

2. Solution Architecture

```
┌──────────────────────────────────────────────────────────────┐
│                      PRESENTATION LAYER                      │
│  ┌──────────────────────────────────────────────────────┐    │
│  │ React.js Frontend (SPA)                              │    │
│  │ • Landing Page   • Authentication   • Search         │    │
│  │ • Booking Flow   • Dashboards       • Management     │    │
│  │ • Bootstrap 5 UI  • Context API State                │    │
│  └──────────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────────┘
                            │
                            │ REST API (Axios)
                            ▼
┌──────────────────────────────────────────────────────────────┐
│                      APPLICATION LAYER                       │
│  ┌──────────────────────────────────────────────────────┐    │
│  │ Node.js + Express.js Server                          │    │
│  │ • Authentication  • Authorization  • Business Logic  │    │
│  │ • API Endpoints   • Validation     • Error Handling  │    │
│  │ • bcrypt Security • CORS Config    • Middleware      │    │
│  └──────────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────────┘
                            │
                            │ Mongoose ODM
                            ▼
┌──────────────────────────────────────────────────────────────┐
│                        DATA LAYER                            │
│  ┌──────────────────────────────────────────────────────┐    │
│  │ MongoDB Database                                     │    │
│  │ • Users Collection     • Flights Collection          │    │
│  │ • Bookings Collection • Relationships & Indexes      │    │
│  └──────────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────────┘
```

3. Core Features by User Role

3.1 Customer Features

🔍 Smart Flight Search

- Search by origin, destination, date
- Past dates automatically disabled
- Return journey option
- Real-time results

🎫 Easy Booking

- 3-step booking process
- Multiple passengers support
- 4 seat class options
- Dynamic price calculation

📊 Booking Management

- View all bookings
- Booking details display
- One-click cancellation
- Status tracking

🔐 Secure Account

- Quick registration
- Instant approval
- Encrypted passwords
- Session management

3.2 Flight Operator Features

✈️ Flight Management

- Add new flights
- Edit flight details
- Set pricing and capacity
- Schedule management

📈 Operator Dashboard

- View all own flights
- Booking statistics
- Passenger information
- Revenue tracking

✅ Approval Workflow

- Registration with verification
- Admin approval required
- Status notifications
- Quality assurance

👥 Booking Insights

- View flight bookings
- Passenger details
- Seat allocation
- Cancellation tracking

3.3 Administrator Features

👤 User Management

- View all users
- User type filtering
- Account status
- Activity monitoring

✅ Operator Approval

- Review operator requests
- Approve/reject workflow
- Verification process
- Quality control

✈️ Flight Oversight

- View all flights
- Cross-operator visibility
- Schedule monitoring
- Data analytics

📊 System Dashboard

- All bookings view
- Platform statistics
- Revenue insights
- Performance metrics

4. Technical Implementation

Component	Technology	Implementation Details
Frontend	React.js 18.2.0	Component-based architecture, React Hooks, Context API for state management, React Router for navigation
UI Framework	Bootstrap 5	Responsive grid system, pre-built components, form controls, mobile-first design
Backend	Node.js + Express.js	RESTful API architecture, middleware support, async/await patterns, error handling
Database	MongoDB 4.4+	Document-based storage, 3 collections (users, flights, bookings), Mongoose ODM
Security	bcrypt	Password hashing with 10 salt rounds, secure authentication, session management
API Client	Axios	Promise-based HTTP requests, interceptors, error handling, JSON transformation

5. Data Model

Collection	Key Fields	Purpose
Users	username, email, usertype, password (hashed), approval	Store user accounts with role-based access control
Flights	flightName, flightId, origin, destination, times, basePrice, totalSeats	Manage flight schedules and availability
Bookings	user (ref), flight (ref), passengers[], totalPrice, journeyDate, seatClass, status	Track all ticket bookings with passenger details

6. Key Workflows

6.1 Customer Booking Workflow

```
1. User Registration/Login
   ↓
2. Search Flights (Origin, Destination, Date)
   ↓
3. View Results (Flight List with Details)
   ↓
4. Select Flight & Seat Class
   ↓
5. Enter Passenger Details (Multiple Passengers)
   ↓
6. Review Total Price (Dynamic Calculation)
   ↓
7. Confirm Booking
   ↓
8. Receive Confirmation (Instant)
   ↓
9. View in Dashboard (Booking Management)
```

6.2 Operator Flight Management Workflow

```
1. Operator Registration
   ↓
2. Wait for Admin Approval
   ↓
3. Login to Dashboard
   ↓
4. Add New Flight (Complete Details)
   ↓
5. Flight Published (Visible to Customers)
   ↓
6. Monitor Bookings (Real-time Updates)
   ↓
7. Edit Flight Details (As Needed)
```

6.3 Admin Approval Workflow

```
1. Admin Login
   ↓
2. View Pending Operator Requests
   ↓
3. Review Operator Details
```

```
   ↓
4. Approve or Reject
   ↓
5. Operator Notified
   ↓
6. Monitor Platform Activity
```

7. Security Implementation

Security Feature	Implementation
Password Security	bcrypt hashing with 10 salt rounds, no plain text storage
Authentication	Session-based with localStorage, secure login/logout
Authorization	Role-based access control (Customer, Operator, Admin)
Input Validation	Client-side and server-side validation, Mongoose schema validation
CORS Protection	Configured allowed origins, preflight request handling
Data Protection	Secure data transmission, no sensitive data in URLs

8. Deployment Strategy

Environment	Configuration	Details
Development	Local	React (3000), Express (6001), MongoDB (27017)
Testing	Staging	Separate database, test data, QA environment
Production	Cloud	Scalable hosting, CDN, database backups, monitoring

9. Success Metrics

Key Performance Indicators (KPIs):

- **User Adoption:** 1000+ registered users in first 3 months
- **Booking Rate:** 90%+ booking completion rate
- **Performance:** < 2 second page load time
- **Satisfaction:** 4.5/5 average user rating
- **Uptime:** 99%+ system availability
- **Security:** Zero data breaches

10. Future Enhancements

- **Phase 2:** Payment gateway integration (Stripe/PayPal)
- **Phase 3:** Email notifications and SMS alerts
- **Phase 4:** Mobile application (React Native)
- **Phase 5:** Advanced analytics and reporting
- **Phase 6:** AI-powered recommendations
- **Phase 7:** Multi-language support

© 2026 SB FlightConnect - Proposed Solution